

Tackling Entangled Enhancement in the Model Pipeline for Autonomous Vehicles

Rashedul Arefin Ifty, Fumio Machida

University of Tsukuba

Tsukuba, Japan

ifty.rashedul@sd.cs.tsukuba.ac.jp, machida@cs.tsukuba.ac.jp

Abstract—Autonomous Vehicles (AVs) are increasingly built as a pipeline of separately trained perception, prediction, and planning models. Each model in the pipeline is often maintained separately when it needs to be updated. However, retraining a single upstream model can cause performance degradation in the downstream models due to distribution shift. Although such a failure is known as entangled enhancement, the phenomenon in the AV pipeline has not been widely investigated. In this paper, we first formally define entangled enhancement in the perception–prediction–planning pipeline by separating a model’s own error from the error inherited from upstream models. We then empirically demonstrate real entangled enhancement in an AV pipeline built from the nuScenes dataset, by analyzing the impact of the retrained perception model across a geographic domain shift on the downstream prediction and planning models. Across 15 retraining updates that vary the dataset size, the training epochs, and the random seed, the perception model improves its perception accuracy every time. However, 6 of them increase the L2 displacement error of the planning model output, exhibiting real entangled enhancement. To proactively mitigate entangled enhancement in the AV pipeline, we propose Cascade-Aware Retraining Assessment (CARA), which guides the decision to adopt the retrained model by measuring the performance difference between the isolated mode and the upstream-dependent mode. Our evaluation with the nuScenes retraining scenario shows that CARA can effectively identify the symptoms of entangled enhancement and can provide a meaningful precaution before adopting the updated model in the pipeline.

Index Terms—autonomous driving, cascade effects, machine learning systems, model pipelines, regression testing, retraining, software reliability

I. INTRODUCTION

Autonomous Vehicles (AVs) are increasingly built as a pipeline of separately trained perception, prediction, and planning models. Perception detects the surrounding agents, prediction forecasts their motion, and planning selects the driving trajectory [1], [2]. In deployment, such a pipeline is maintained under domain drift one model at a time. A perception model is retrained for a new city, a prediction model is updated when traffic changes, and a planning model is revised for new regulations. This kind of maintenance is possible only in a pipeline. Each model can be retrained on its own. In an end-to-end network the whole network must be retrained at once. We therefore adopt the pipeline as the setting for this study.

In practice, one model is usually retrained while the others stay fixed. This is where a problem can appear. The retrained

model becomes better on its own metric, but the models after it become worse. The reason is that those models were built and evaluated on the output of the earlier version of the retrained model. The first reported instance of this self-defeating behavior appeared in an object-detection setting [3], where a better detector reduced the accuracy of the downstream task. Wang and Machida [4] modeled and analyzed entangled enhancement in a two-model machine learning system by a continuous-time Markov chain. Entangled enhancement has also been reported in a sentence-classification system [5]. Sculley et al. [6] describe the same difficulty more generally. When learning components depend on each other, changing one of them creates maintenance problems for the others. However, no existing study examines entangled enhancement across an entire perception–prediction–planning pipeline for AVs. Although previous studies also consider the perception module of an AV, their analysis was limited to two sequential models and focused only on the perception component.

To address this gap, this paper studies how a single-model retraining event propagates through an AV pipeline, using empirical analysis. For the empirical analysis, we build a three-model pipeline on the nuScenes dataset [7]. The perception model is YOLOv11n [8], an object detector applied to the front-camera image. The prediction model is a constant-velocity trajectory predictor. The planning model is a rule-based planner from the Intelligent Driver Model (IDM) family [9], whose proposal-and-score design follows PDM-Closed [10]. We train the perception model on Boston scenes and fine-tune it on Singapore scenes, a well-known domain gap within nuScenes, and investigate how the updates impact the downstream performance. To separate a model’s own error from the error it inherits from upstream, each downstream model is measured in two modes, one with ground-truth inputs and one with the live upstream output. Because a single retraining is only one data point, we run a campaign of 15 retraining updates that vary the amount of fine-tuning data, the number of training epochs, and the random seed.

This paper makes three contributions.

- We formally define entangled enhancement in the perception–prediction–planning pipeline for AVs, and introduce the cascade coupling factor that quantifies the intensity of entangled enhancement.
- We empirically investigate real entangled enhancement

in an AV pipeline generated from the nuScenes dataset, showing that six out of 15 retraining trials exhibit entangled enhancement.

- We propose a retraining assessment framework, CARA, which can evaluate potential entangled enhancement by measuring the performance difference between the isolated mode and the upstream-dependent mode.

The remainder of the paper is organized as follows. Section II formalizes the pipeline model and the definition of entangled enhancement. Section III presents the proposed CARA method. Section IV details the experimental setup and the retraining campaign. Section V reports the empirical evidence of entangled enhancement and the evaluation of CARA. Section VI discusses the mechanism and its implications, Section VII reviews related work, and Section VIII concludes.

II. SYSTEM MODEL AND METHODOLOGY

A. Formal Pipeline Model

We model an AV pipeline as a composition of three functions,

$$S = C_1 \circ C_2 \circ C_3, \quad S(x) = C_3(C_2(C_1(x))), \quad (1)$$

where x is an input camera frame processed through the three models:

- **Perception** (C_1). The output $z_1 = C_1(x)$ is a list of the objects the model found in the image. For each object it gives the type, the size, and the position on the road around our own car.
- **Prediction** (C_2). The output $z_2 = C_2(z_1)$ is, for every detected object, a forecast trajectory over the next several seconds.
- **Planning** (C_3). The output $z_3 = C_3(z_2)$ is the ego vehicle's planned trajectory, a sequence of positions over a 3 s horizon.

The outputs z_1 and z_2 connect one model to the next. A change in C_1 therefore changes z_1 , which is the input of C_2 . The change then reaches z_2 , which is the input of C_3 . Now suppose that only C_1 is retrained. The parameters of C_2 and C_3 are unchanged. Any change in their outputs therefore comes only from their inputs.

B. Entangled Enhancement

Entangled enhancement occurs when an upstream model is retrained and improves its own performance, and yet the system as a whole degrades [4]. This entanglement happens because the downstream models were tuned to the statistics of the *original* upstream output and are shifted after the upstream model update. We denote by δ_1 the improvement of the perception performance of C_1 after retraining, and by Δ_3 the degradation of the planning performance of C_3 . Entangled enhancement is formally specified by the joint condition

$$\underbrace{\delta_1 > 0}_{C_1 \text{ improves}} \wedge \underbrace{\Delta_3 > 0}_{\text{system degrades}}. \quad (2)$$

The condition requires the upstream model to improve on its *own* performance ($\delta_1 > 0$) while the system nonetheless degrades ($\Delta_3 > 0$). The upstream model must improve for this to be entangled enhancement. If the upstream model became worse and the system became worse with it, that is an ordinary regression.

C. Cascade Gap and the Isolation Property

To make entangled enhancement measurable, we separate a model's own error from the error inherited from the upstream models. Let z_k denote the output of model C_k , and let $M_k(z_k)$ be an error metric evaluated on that output, with lower values better. We evaluate each downstream model in two modes. In *isolated* mode a model receives ground-truth inputs, and its output is written z_k^{iso} . For C_2 and C_3 ,

$$z_2^{\text{iso}} = C_2(z_1^*), \quad z_3^{\text{iso}} = C_3(z_2^*), \quad (3)$$

where z_k^* is the ground-truth value of C_k . In *pipeline* mode a model receives the direct output of the upstream neighbor, and its output is written z_k^{pipe} ,

$$z_2^{\text{pipe}} = C_2(C_1(x)), \quad z_3^{\text{pipe}} = C_3(C_2(C_1(x))). \quad (4)$$

The metric in isolated mode, $M_k(z_k^{\text{iso}})$, reflects the model's own error, while the metric in pipeline mode, $M_k(z_k^{\text{pipe}})$, reflects its own error together with the error inherited from the upstream model. The difference between the two is the inherited part, which we call the *cascade gap* at C_k ,

$$\Gamma_k = M_k(z_k^{\text{pipe}}) - M_k(z_k^{\text{iso}}). \quad (5)$$

Isolated mode needs ground-truth inputs. It can therefore be run only in an experiment, not in a real vehicle. It shows whether a change came from the model itself or from its input.

Consider a retraining event that replaces an upstream model C_k by an updated model C'_k , whose outputs we write z'_k . If the pipeline is properly composed, the retraining leaves the isolated-mode output of every downstream model that was not retrained unchanged, because that output depends only on the ground-truth inputs. We call this the *isolation property*,

$$M_k(z_k^{\text{iso}}) = M_k(z_k^{\text{iso}'}) \quad (6)$$

Equation (6) is a testable property of the setup rather than an assumption. Any change in the isolated-mode metric of a model that was not retrained indicates contamination, and we verify the equation in Section V. The effect that the retraining causes at model C_k is the change in its pipeline metric,

$$\Delta_k = M_k(z_k^{\text{pipe}'}) - M_k(z_k^{\text{pipe}}), \quad (7)$$

which, given (6), equals the change in the cascade gap Γ_k .

D. Coupling Factor

To quantify the intensity of entangled enhancement, we relate the system performance degradation Δ_3 to the upstream improvement δ_1 , through the *cascade coupling factor*

$$\rho_{1 \rightarrow 3} = \frac{\Delta_3}{\delta_1}. \quad (8)$$

Under the sign convention above, a positive δ_1 improves the upstream model and a positive Δ_3 worsens the system. Entangled enhancement therefore corresponds to $\rho_{1 \rightarrow 3} > 0$, and its magnitude measures how much regression of the planning metric accompanies a unit of upstream improvement. The factor is well defined whenever $\delta_1 \neq 0$. As Section V shows, the retraining campaign enters the entangled-enhancement regime in 6 of 15 updates, for which $\rho_{1 \rightarrow 3} > 0$ is read together with the full effect vector (Δ_2, Δ_3) rather than as an isolated scalar.

III. CASCADE-AWARE RETRAINING ASSESSMENT

The formal definition of Section II exposes a concrete hazard for maintenance. An updated model that improves on its own metric, and even improves the metric of its immediate consumer, can still degrade the planning output. Neither a check on the updated model alone nor a check on the next model alone is therefore a sufficient criterion for admitting the update. We turn the definition into an admission check for model updates, Cascade-Aware Retraining Assessment (CARA). CARA is an explicit gate that measures the effect of an update at every downstream model and admits the update only when the effect on the planning metric stays within a threshold. Fig. 1 summarizes the decision and Algorithm 1 states it.

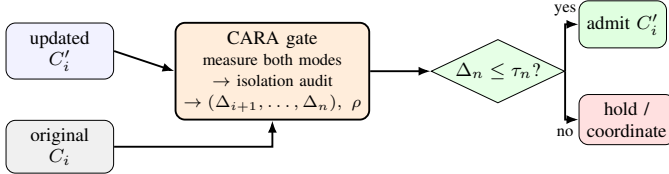


Fig. 1: The CARA admission check for an updated model.

A. The Admission Procedure

The inputs to CARA are a pipeline $C_1 \rightarrow \dots \rightarrow C_n$, an audit set A , a candidate update that replaces the original model C_i by an updated model C'_i , and a threshold τ_n on the metric of the last model. The audit set is a fixed collection of driving scenes, annotated with the ground truth that the isolated mode requires. Holding A fixed makes the two evaluations comparable, because C_i or C'_i is then the only difference between them. CARA produces an admit-or-hold decision, the change at every downstream model, and the coupling factor. The procedure has three steps, shown in Fig. 1 and stated in Algorithm 1.

The first step measures both models on the same audit set. CARA runs the two-mode measurement of Section II-C on A with C_i in place, then repeats it with C'_i in place under the same seeds. Each run records, for every downstream model, its metric in isolated mode and its metric in pipeline mode.

The second step checks the isolation property (6). For every downstream model C_k with $k > i$, which was not retrained, the isolated-mode metric must be identical before and after the update, because that metric depends only on ground-truth

inputs. A difference indicates leakage or nondeterminism and makes the assessment invalid. This check is what allows any measured change to be attributed to the update.

The third step computes the change at each model and decides. For every downstream model C_k , CARA computes the change in its pipeline-mode metric Δ_k from (7). CARA then reports all of these changes together, as $(\Delta_{i+1}, \dots, \Delta_n)$. Two neighbouring models can move in opposite directions, which a single summary number would hide. CARA also reports the coupling factor $\rho_{i \rightarrow n} = \Delta_n / \delta_i$, whose sign identifies entangled enhancement under the convention of Section II-B. The update is admitted only when the effect on the last model stays within the threshold, $\Delta_n \leq \tau_n$, and the collision rate does not become worse. Otherwise the update is held, and the downstream consumers are retrained together with it.

Algorithm 1: Cascade-Aware Retraining Assessment (CARA)

Input: pipeline $C_1 \rightarrow \dots \rightarrow C_n$, audit set A , original model C_i , updated model C'_i , threshold τ_n

Output: admit or hold decision, the changes $(\Delta_{i+1}, \dots, \Delta_n)$, coupling factor ρ

Measure both modes with C_i on A , giving $M_k(z_k^{\text{iso}})$ and $M_k(z_k^{\text{pipe}})$ for every downstream C_k

Measure both modes with C'_i on A under the same seeds, giving $M_k(z_k^{\text{iso}'})$ and $M_k(z_k^{\text{pipe}'})$

foreach downstream C_k with $k > i$ **do**

if $M_k(z_k^{\text{iso}}) \neq M_k(z_k^{\text{iso}'})$ **then**

return invalid // isolation violated

end

end

$\Delta_k \leftarrow M_k(z_k^{\text{pipe}'}) - M_k(z_k^{\text{pipe}})$ for $k = i+1, \dots, n$

$\rho \leftarrow \Delta_n / \delta_i$

if $\Delta_n \leq \tau_n$ **and** collision rate not worse **then**

return admit C'_i

else

return hold and retrain downstream models

end

B. Low-Cost Drift Screen

The admission procedure runs the entire pipeline on the audit set, at the cost of a complete evaluation. We precede it with a lower-cost screen that reads only the output of the updated perception model. The screen produces a single number, which we call the *drift score*. The drift score measures how differently the updated detector behaves compared to the original detector on the same scenes, and not whether the updated detector is better or worse. The screen compares the detections of the updated model C'_1 against those of the original model C_1 on the same audit scenes. For the continuous features of a detection, namely the per-image count, the confidence, and the box-center position, the screen takes the Kolmogorov–Smirnov statistic between the two

models. For the categorical class label, it takes the Jensen–Shannon divergence between the two class histograms. These per-feature statistics all lie in $[0, 1]$, and the screen averages them into the drift score, which therefore also lies in $[0, 1]$. The construction follows the between-stage two-sample tests used in production data-validation systems [11]. The screen is model-free, because the computation consumes only the detection records that the pipeline already emits. A large drift score marks an update whose output moved far from the original, and the screen uses it to decide which updates warrant the full admission procedure. The drift score is a screening statistic and not the admission criterion. The admission decision uses the change Δ_n at the last model, which Algorithm 1 computes.

C. Applicability to Pipeline Architectures

Admission never rests on the metric δ_i of the updated model, because entangled enhancement is exactly the case in which δ_i improves while the planning output degrades. CARA instead reports the change at every downstream model, which prevents an improvement at one model from hiding a regression further downstream. The isolation check detects contamination by an exact-equality test on the models that were not retrained, and that test is available only because the models are separable in a pipeline. The method is therefore constructible only in a pipeline architecture. An end-to-end network has no separate models to leave unchanged, no isolated mode, and no points at which the change Δ_k could be measured. The cost of CARA is inference on a fixed audit set, with no training.

IV. EXPERIMENTAL SETUP

A. Dataset and Components

We use the full nuScenes v1.0 trainval split [7]. A nuScenes *scene* is a ≈ 20 s driving clip, annotated at 2 Hz. Each of these annotated samples is a *keyframe*. A scene therefore contributes about 40 keyframes, and we use the front-camera (CAM_FRONT) keyframe as one evaluation frame throughout. The trainval split contains 850 scenes (34,149 keyframes in total, ≈ 45 GB), of which 467 scenes were recorded in Boston and 383 in Singapore. Partitioning by recorded location and splitting each city 60/40 into train and validation gives `boston_train/boston_val` (280/187 scenes) and `singapore_train/singapore_val` (229/154 scenes). The Boston detector is trained on the 11,087 CAM_FRONT keyframes of `boston_train`. To keep evaluation affordable across the many fine-tunes of the campaign while spanning the whole city, all cascade measurements use a fixed subset of `singapore_val` obtained by an even stride over its 154 scenes down to 50 scenes, comprising $n = 2,020$ annotated keyframes. This same 50-scene audit set is reused, unchanged, for every one of the 15 updates, which keeps the comparisons exact. For C_1 training, the 3D box annotations are projected onto the front-camera keyframes to produce 2D labels over the eight driving classes that the pipeline consumes, namely car, truck, bus, trailer, construction vehicle, motorcycle, bicycle, and pedestrian.

We employ YOLOv11n [8] as the perception model C_1 , an object detector applied to the front-camera image. We fine-tune C_1 for 10 epochs on `boston_train` from COCO weights to obtain the original model. To generate an updated model, the same YOLOv11n is fine-tuned further on `singapore_train` starting from the Boston checkpoint. At inference, each 2D detection is lifted to an ego-frame ground position by intersecting its bottom-center pixel ray with the ground plane, which supplies the agent positions z_1 that C_2 consumes. For the prediction model C_2 we adopt a constant-velocity motion model, which is deterministic and carries no trainable parameters. In isolated mode it rolls agents out at their ground-truth velocity, and in pipeline mode it anchors a zero-velocity rollout at the single-frame detections of C_1 . Any change in its pipeline output is therefore attributable solely to a change in C_1 . The Intelligent Driver Model [9] is used for the planning model C_3 , which is a rule-based planner of the proposal-and-score type. The planner works in two stages. In the proposal stage the planner enumerates a finite set $\mathcal{P}$ of candidate trajectories, obtained by sweeping target speeds and lateral offsets around the current lane. In the scoring stage it gives each candidate a cost made of two terms. The first term, $J_{\text{prog}}(\pi)$, is a progress cost that penalises a candidate for advancing slowly along the route. The second term is a safety cost that penalises a candidate for passing close to any predicted agent. For an agent a in the predicted futures z_2 , let $d(\pi, a)$ be the closest distance between candidate π and that agent over the next 3 seconds. The penalty ϕ is a decreasing function of this distance, and a candidate that passes near an agent therefore receives a large penalty, and the weight λ sets how strongly safety is traded against progress. The executed plan is the candidate of minimum total cost,

$$\pi^* = \arg \min_{\pi \in \mathcal{P}} J_{\text{prog}}(\pi) + \lambda \sum_{a \in z_2} \phi(d(\pi, a)). \quad (9)$$

This design reproduces the PDM-Closed [10] architecture without its learned scorer. The safety term is evaluated on z_2 , and that dependence is the channel through which a perturbed detection reaches the plan. A detection that moves, appears, or disappears changes z_2 , hence the safety cost, hence which candidate wins.

B. Retraining Procedure

Each retraining event follows the same procedure, and only C_1 is ever changed.

- 1) Train C_1 on the source domain (Boston scenes), and freeze C_2 and C_3 permanently.
- 2) Measure both modes on the target-domain validation set (Singapore). The measurement records C_1 detection quality (mAP@50), the minADE of C_2 in isolated and pipeline mode, and the L2@{1, 2, 3}s of C_3 in isolated and pipeline mode.
- 3) Fine-tune C_1 on the target domain (Singapore scenes) to obtain the updated model C'_1 .
- 4) Repeat the identical measurement with C'_1 in place.

- 5) Verify the isolation property (6) and compute the changes (7) at C_2 and C_3 .

The Boston-to-Singapore shift within nuScenes is a genuine and well-studied domain gap, with left-hand versus right-hand traffic, different vehicle and agent mixes, and different lighting conditions [7].

C. Metrics

C_1 is scored with standard detection metrics (mAP@50, mAP@50-95, precision, recall) on the target-domain validation images. For C_2 , the metric is the minimum average displacement error (minADE). For an agent with forecast positions $\hat{p}_t^{(m)}$ under mode m and ground-truth positions p_t^* over T future steps,

$$\text{minADE} = \min_m \frac{1}{T} \sum_{t=1}^T \|\hat{p}_t^{(m)} - p_t^*\|_2, \quad (10)$$

averaged over agents. Our deterministic predictor has a single mode, and the min therefore collapses to that mode. For C_3 , the metric is the L2 displacement error between the planned ego trajectory $\hat{e}_t$ and the human driver's recorded trajectory e_t^* , measured τ seconds ahead of the current frame,

$$\text{L2@}\tau = \|\hat{e}_\tau - e_\tau^*\|_2, \quad \tau \in \{1, 2, 3\} \text{ s}, \quad (11)$$

reported together with the collision rate. For the campaign we also report a *plan shift*, the distance between the plan produced with the original C_1 and the plan produced with the updated C_1' on the same scene, which measures how far a retraining moves the driving decision independent of the human path. The L2 metric is the standard open-loop planning protocol of nuScenes, which earlier studies on end-to-end driving models also use [2], [12], [13]. This protocol has known limitations. We use it differentially, comparing two runs that differ only in C_1 , and we discuss what this comparison does and does not show in Sections V and VII.

D. The Retraining Campaign

We run a campaign of 15 **retraining updates** of C_1 , each measured through the identical measurement of Section IV-B. The campaign varies three factors.

- **Fine-tuning data size.** We fine-tune C_1 on three amounts of Singapore data, namely the full `singapore_train` set of 229 scenes, one half of it, and one quarter of it. The half and the quarter are obtained by taking every second and every fourth scene of a fixed scene ordering, which makes the same subsets available to every run. A smaller set gives the detector less data for adapting to the new city. Varying this amount therefore tests whether the downstream effect depends on how much the detector was able to adapt.
- **Training length.** We use 5, 10, and 20 epochs, to test whether longer fine-tuning changes the downstream effect.
- **Random seed.** We repeat each setting with three seeds, which separates a result from seed noise.

The matrix combines three data sizes at 10 epochs with two additional training lengths (5 and 20) at the full data size, each over three seeds, giving $3 \times 3 + 2 \times 3 = 15$ updates. The original C_1 (Boston-trained) and the unchanged C_2 and C_3 are shared across all 15 updates, and only the Singapore fine-tuned C_1' changes between them. Within each update the fine-tuning run uses a single fixed random seed, and the evaluation of C_2 and C_3 is deterministic because neither model has trainable parameters. The isolation property (6) can therefore be checked for exact equality rather than for statistical closeness.

E. Measurement Protocol

For each of the 15 updates we record the metrics of Section IV-C on `singapore_val`, once with C_1 in place and once with C_1' in place. From the two records we compute δ_1 , Δ_2 , Δ_3 , the plan shift, and whether the update meets the condition for entangled enhancement (Section II-B).

V. EVALUATION RESULTS

The evaluation pursues the two empirical goals of this paper. The first goal is to establish evidence of entangled enhancement in the AV pipeline, which is the second contribution. The second goal is to evaluate CARA on the same updates, which is the third contribution.

Eight experiments serve these goals, and each has a distinct objective. The *reference update* (Section V-A) asks whether entangled enhancement occurs at all in a single update. The *detector analysis* (Section V-B) asks whether the planning degradation could instead be explained by a weaker detector. The *isolation control* (Section V-C) asks whether the observed change can be attributed to the update rather than to the measurement setup. The *cascade location analysis* (Section V-D) asks where along the pipeline the change caused by the retraining is largest. The *campaign* (Section V-E) asks how often, and how strongly, entangled enhancement occurs across 15 updates rather than one. The *class-balance study* (Section V-F) asks whether the effect is an artifact of class imbalance in the training data. Finally, the *drift screen evaluation* (Section V-G) and the *admission-rule evaluation* (Section V-H) ask whether the low-cost screen of CARA predicts the downstream effect and whether its decision rule separates the harmful updates from the rest.

A. The Reference Update

Objective. Determine whether a single retraining of the perception model produces entangled enhancement as defined in (2).

The reference update is the full-data, 10-epoch, seed-1 fine-tune. Table I reports its measurements on `singapore_val` ($n = 2,020$ frames). The update satisfies every clause of the definition. The detector improves on its own metric ($\delta_1 = +0.119$, mAP@50 $0.137 \rightarrow 0.256$), the input it gives to C_2 improves ($\Delta_2 = -2.17$ m, a 27% reduction), the isolation property holds, and *yet* the trajectory-agreement metric of the planner degrades ($\Delta_3 = +0.124$ m, from 5.55 to 5.67 m).

The update therefore exhibits entangled enhancement in an AV pipeline, and it is a controlled, quantified instance of the correction-cascade pattern that the technical-debt literature describes only qualitatively [6]. The metric of C_3 in isolated mode remains fixed at 4.64 m, because the planner receives identical ground-truth inputs in both runs, while its metric in pipeline mode moves from 5.55 to 5.67 m. Fig. 2 shows these two pairs of values.

The two metrics of C_3 moreover move in opposite directions. The collision rate improved, from 2% under the original model to zero, while the trajectory-agreement metric worsened over the same update. A better detector therefore pushes the two readings in opposite directions, which is precisely why a single-metric gate on the system output is unsafe and why CARA reports the change at every downstream model.

TABLE I: Reference update measured on Singapore validation ($n = 2,020$ frames). “Before” is the Boston-trained C_1 and “after” is the Singapore-fine-tuned C_1' .

Metric	Before	After	Δ	$\Delta\%$
C_1 mAP@50	0.1370	0.2556	+0.1186	+86.6
C_2 minADE, iso. (m)	1.1211	1.1211	0.0000	0.00
C_2 minADE, pipe. (m)	7.9310	5.7590	-2.1720	-27.4
C_3 L2@3s, iso. (m)	4.6384	4.6384	0.0000	0.00
C_3 L2@3s, pipe. (m)	5.5463	5.6703	+0.1240	+2.2
C_3 collision rate, pipe.	0.020	0.000	-0.0200	n/a

B. The Detector Improves, Yet the System Degrades

Objective. Rule out the explanation that the planning degradation is caused by a weaker detector rather than by the cascade.

The effect of the reference update is not caused by a weaker detector. The aggregate metric of C_1 rises by 86.6%, and the 229-scene Singapore fine-tune set is large enough that adaptation to the new city genuinely improves the detector rather than over-fitting it. The confusion matrix of the Boston-trained detector (Fig. 3) shows the dominant car class well separated, with the rarer classes still frequently confused with background. That residual class imbalance motivates the study of Section V-F. Because the detector improved on its own metric, the planning degradation cannot be attributed to a weaker perception model, and it is therefore a genuine instance of a better model disturbing the system it feeds.

C. The Isolation Control

Objective. Confirm that the measured change is attributable to the update itself and not to leakage or nondeterminism in the measurement setup.

Before any of these changes is interpreted, C_2 and C_3 must be shown to be untouched. The check validates the measurement setup rather than reporting a result. The three isolated-mode rows of Table I are exactly zero-delta. The metric of C_2 in isolated mode is 1.1211 m in both runs, and the metric of C_3 in isolated mode is 0.7458/2.4100/4.6384 m

at 1, 2, and 3 seconds in both runs, to every recorded digit. For the deterministic, parameter-free C_2 and C_3 fed identical ground-truth inputs, this invariance is expected by construction, and any nonzero delta would have signalled state or data leakage between the runs. The check therefore certifies that the pipeline-mode deltas in Table I arise solely from the retraining of C_1 , transmitted through the model inputs. The exact-equality check held across all 15 updates of the campaign.

D. Where the Cascade Concentrates

Objective. Locate the point along the pipeline at which the change caused by the retraining is largest.

The largest single effect in Table I occurs at the step from C_1 to C_2 . With the Boston-trained C_1 , the metric of C_2 in pipeline mode was 7.9310 m against an isolated-mode floor of 1.1211 m, a cascade gap Γ_2 of 6.81 m. This gap conflates two sources, namely the detection error of C_1 and the loss of velocity information, because C_2 uses ground-truth velocity in isolated mode whereas it uses a zero-velocity rollout in pipeline mode (Section IV). It therefore bounds the combined effect rather than the perception error alone. After the Singapore fine-tune, the metric of C_2 in pipeline mode fell to 5.7590 m ($\Gamma_2 = 4.64$ m), an improvement of 27.39% obtained without modifying C_2 . The change $\Delta_2 = -2.17$ m is attributable to C_1 alone, because the velocity-information gap is constant across the two runs. The first downstream model therefore receives the largest change caused by the retraining.

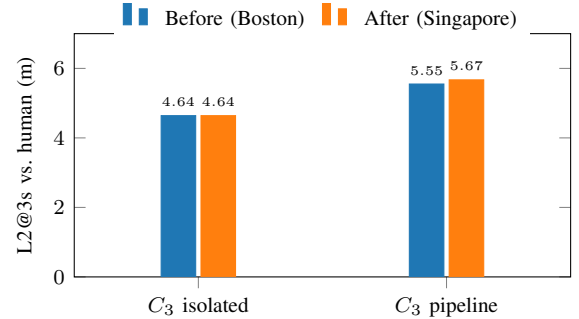


Fig. 2: Cascade signature on planning. The metric of C_3 in isolated mode stays fixed while its metric in pipeline mode moves.

E. The Campaign

Objective. Determine how often entangled enhancement occurs, and how strongly, when the retraining configuration is varied rather than fixed.

The reference update is a single point in a large space of possible updates. Table II reports all 15 updates of the campaign (Section IV-D), and three patterns emerge.

First, the detector improves on its own metric in every update, with $\delta_1 > 0$ in all 15, and prediction improves as well, with $\Delta_2 < 0$ in all 15. Fine-tuning on the new city genuinely sharpens C_1 across the whole campaign.

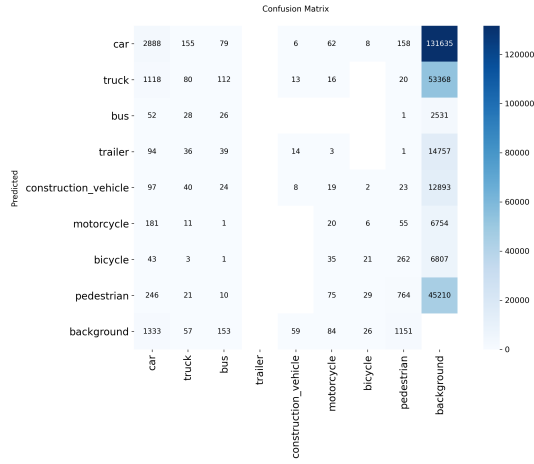


Fig. 3: Confusion matrix of the Boston-trained C_1 on full trainval.

Second, and most importantly, this is enough to observe *entangled enhancement directly*: in 6 of the 15 updates $\delta_1 > 0$ while $\Delta_3 > 0$, i.e. the detector improves yet the planner degrades. The six span a range of severities, from a marginal $+0.008$ m up to a material $+0.193$ m. No class re-balancing is required to reach this condition, which arises in the ordinary campaign.

Third, the direction of the effect on planning is campaign-dependent. The planner is worse ($\Delta_3 > 0$) in 6 updates and better ($\Delta_3 < 0$) in the other 9. The training configuration does not predict which updates degrade the planner. A rule based on the detector alone is therefore not sufficient, and the update has to be checked at the downstream models (Section III).

TABLE II: The 15-update retraining campaign on singapore_val ($n = 2,020$ frames). A positive δ_1 marks an improved detector, a negative Δ_2 an improved prediction, and a positive Δ_3 a worse planner. The shift column is the plan shift in metres. “ent. enh.” marks entangled enhancement ($\delta_1 > 0$ with $\Delta_3 > 0$) and “helped” marks $\Delta_3 < 0$. Every update has $\delta_1 > 0$.

Update	data	δ_1	Δ_2	Δ_3	shift	outcome
Full, 10 ep, s1	full	+0.119	-2.17	+0.124	1.05	ent. enh.
Full, 10 ep, s2	full	+0.122	-2.28	+0.193	1.02	ent. enh.
Full, 10 ep, s3	full	+0.129	-2.15	+0.187	0.92	ent. enh.
Half, 10 ep, s1	half	+0.114	-2.55	-0.121	1.35	helped
Half, 10 ep, s2	half	+0.120	-2.71	-0.230	1.19	helped
Half, 10 ep, s3	half	+0.106	-1.77	-0.151	1.46	helped
Quarter, 10 ep, s1	qtr	+0.121	-2.71	+0.073	0.79	ent. enh.
Quarter, 10 ep, s2	qtr	+0.113	-1.29	-0.093	0.78	helped
Quarter, 10 ep, s3	qtr	+0.101	-2.82	+0.008	1.00	ent. enh.
Full, 5 ep, s1	full	+0.138	-1.08	-0.095	0.89	helped
Full, 5 ep, s2	full	+0.122	-1.88	-0.130	1.22	helped
Full, 5 ep, s3	full	+0.130	-1.08	-0.041	0.92	helped
Full, 20 ep, s1	full	+0.142	-2.59	+0.037	1.37	ent. enh.
Full, 20 ep, s2	full	+0.126	-1.77	-0.083	1.50	helped
Full, 20 ep, s3	full	+0.130	-2.44	-0.020	1.20	helped

F. Robustness to Class Imbalance

Objective. Test whether entangled enhancement is an artifact of class imbalance in the training data rather than a property of the pipeline.

The campaign already yields entangled enhancement without any intervention on the training data. A natural question is whether the effect is an artifact of the residual class imbalance visible in Fig. 3, where rare classes are still confused with background. To test this, we repeat the campaign with a class-balanced training split. Images containing rare classes are oversampled so that the per-class instance distribution flattens, with a repeat factor $(n_{\max}/n_c)^p$ per class c clipped to $[1, 12]$, while the validation split is left untouched so metrics stay comparable. We run two balance strengths ($p \in \{0.5, 1.0\}$), two training lengths (10 and 20 epochs), and three seeds, giving 12 updates.

The effect persists under class balancing. Across the 12 updates the detector still improves in every case ($\delta_1 > 0$ in 12/12), and entangled enhancement appears in 4 of the 12. Three occur at $p=0.5$ (ep10_s3, ep20_s1, ep20_s2, with δ_1 up to $+0.143$ and Δ_3 up to $+0.139$ m) and one at $p=1.0$ (ep20_s1, $\delta_1 = +0.125$, $\Delta_3 = +0.075$ m). Entangled enhancement appears at both balance strengths and at both training lengths, with three of the four cases occurring at 20 epochs. Rebalancing the rare classes therefore does not remove the effect. The condition is not an artifact of the residual class imbalance but a property of the pipeline under domain-shift retraining.

G. Evaluation of CARA’s Drift Screen

Objective. Determine whether the low-cost drift score of CARA predicts the downstream effect that only the full pipeline reveals.

For each of the 15 updates we compute the drift score of the updated detector against the original detector on the audit scenes. Ranked against the plan shift, the score is a moderate predictor of *how much* the plan moves, with a rank correlation of ≈ 0.56 (Fig. 4). The score does not separate the updates that worsen the planning metric ($\Delta_3 > 0$) from the rest. Its AUROC for this classification is ≈ 0.48 , which is the value expected from a random ranking. Every retrained detector improved consistently, and all scores therefore fall in a narrow band (0.05 to 0.08) in which the two groups interleave. The drift screen therefore indicates the *magnitude* of downstream disturbance but not its *direction*, which makes it a filter that prioritizes updates for the full admission procedure of Algorithm 1 rather than a standalone rule.

H. Evaluation of CARA’s Admission Rule

Objective. Determine whether the drift score, used as a binary admit-or-hold rule, separates the updates that regress the planning metric from the rest.

We label an update “L2-regressing” when it worsened the trajectory-agreement metric ($\Delta_3 > 0$). Of the 15 updates, 6 are L2-regressing and 9 are not.

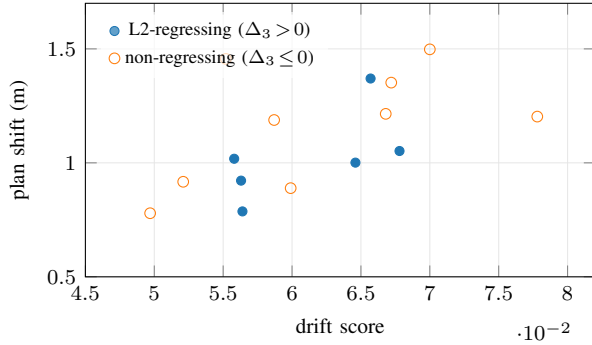


Fig. 4: Drift score vs. plan shift over the 15 updates. Drift tracks the magnitude of plan movement, but the two groups interleave.

As a thresholded decision the rule is weak, as Table III shows. At a conservative threshold every update is held, and no L2-regressing update is therefore admitted (recall 1.0) but no update is admitted at all (specificity 0), which is safe but provides no discrimination. The best balanced operating point still catches every L2-regressing update (recall 1.0) but admits only two of nine non-regressing ones (specificity 0.22), at precision 0.46. The drift screen can be tuned to catch every L2 regression, but at the cost of holding most other updates too. The two planning clauses of Algorithm 1 disagree here, which is the point. The original pipeline had a 2% collision rate, and every updated model drove it to zero, and the collision-rate clause would therefore admit all 15. The L2 clause, in contrast, holds the six $\Delta_3 > 0$ updates. CARA holds those six on L2 even though their collision rate improved, and neither clause alone is a safe gate. CARA does not reduce the assessment to a single admit-or-hold decision. CARA reports the change at every downstream model, and the disagreement between the two metrics of C_3 is therefore visible to the maintainer, who can then decide whether to hold the update or to retrain the downstream models together with it.

TABLE III: CARA drift-rule decisions against ground truth ($\Delta_3 > 0 = \text{L2-regressing}$), on the 15 full-trainval updates (6 L2-regressing, 9 not). The held-out row tunes the threshold on a 7-update tune split and reports on the untouched 8-update test split.

Threshold	TP/FP/FN/TN	Rec.	Spec.	Prec.	F1
Strict (hold all)	6/9/0/0	1.00	0.00	0.40	0.57
Balanced	6/7/0/2	1.00	0.22	0.46	0.63
Held-out (test)	3/3/0/2	1.00	0.40	0.50	0.67

We interpret these results conservatively. The threshold is chosen on a 7-update tune half and applied to the untouched 8-update test half. The reported operating point is therefore not fitted on the data it is scored against. On that held-out test the rule still catches every L2-regressing update (recall 1.0) but keeps a low specificity (0.40), confirming that the drift screen is a prioritization filter, not a decision rule. With only

8 test updates the estimates are wide: Wilson 95% intervals are $[0.44, 1.00]$ recall, $[0.12, 0.77]$ specificity, and $[0.19, 0.81]$ precision, and they are indicative rather than tight. The full admission procedure of Algorithm 1 remains the reliable decision, and the drift screen serves as its low-cost first filter that never misses an L2-regressing update.

I. Training Diagnostics

Both fine-tunes converged within their epoch budgets with monotone loss decay, as Fig. 5 shows for the Boston run. The box, classification, and distribution-focal losses decrease on both the train and validation splits, while precision, recall, and mAP@50 increase, with mAP@50 rising from 0.116 at epoch 1 to 0.137 at epoch 10. A single update, comprising the fine-tune and both evaluations, costs about 24 GPU-minutes, and the whole 15-update campaign ran in about six GPU-hours. That budget is far below the cost of training an end-to-end planner [14], which is what makes a campaign of repeated retraining feasible.

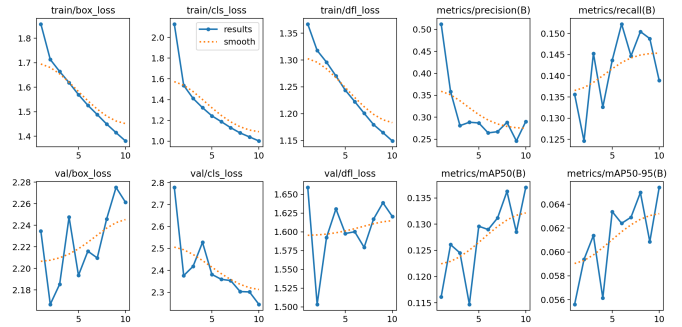


Fig. 5: YOLOv11n C_1 Boston fine-tune training curves.

VI. DISCUSSION

We explain the mechanism behind the non-monotone cascade and place CARA among related maintenance techniques.

A. Why the Cascade Is Non-Monotone

Under the Boston-trained C_1 , the planner's inputs were substantially displaced ($\Gamma_2 = 6.81$ m) in a regime whose errors the proposal-and-score rules absorbed. Missed or grossly misplaced agents leave the proximity penalty inactive, and the planner therefore follows its kinematic prior, which on these scenes approximated the human trajectory. The updated C_1 relocated detections near their true positions and re-activated the penalty with correctly anchored but coarse (zero-velocity) forecasts, causing the planner to react to obstacles it had previously ignored. Downstream models can thus absorb upstream error, and, symmetrically, upstream improvement can expose masked downstream sensitivity. An updated model therefore cannot be admitted on its own output alone.

This mechanism also cautions against a literal interpretation of the planning metric. The collision rate *fell*, from 2% under the original model to zero after retraining, even as L2 rose,

and the higher L2 of the reference case arises because the improved detector caused the planner to react to real agents rather than follow a kinematic prior that happened to track the recorded human path. That behaviour is the ego-forecasting shortcut of nuScenes open-loop L2, under which perception-agnostic prediction is competitive [13] and open-loop error is weakly tied to closed-loop quality [15]. A regression on this metric may therefore mark a *safer* planner that scores worse on a measure rewarding ignored perception, and the planning metrics here should therefore be read as reduced trajectory agreement, not an established safety regression, and a closed-loop or NAVSIM-style metric [16] is the appropriate safeguard.

B. CARA Among Related Maintenance Techniques

The findings give three maintenance rules that CARA (Section III) enforces directly. An update must not be admitted on the updated model’s own metric, and admission must not stop at the next model, because the entangled-enhancement cases are exactly those in which C_1 improved and C_2 improved while C_3 degraded. A maintenance policy should also assess several candidate updates rather than trust one, because the effect on planning varies across updates.

CARA is one of several complementary techniques for these rules, not a replacement for them. Regression-constrained retraining such as positive-congruent training [17] and backward-compatible training [18] *reduces* the cascade at its source by constraining the update. Uncertainty-propagating interfaces [19] *absorb* the residual by passing distributions instead of point estimates across the perception–prediction boundary. Between-stage data validation [11] *monitors* the interface in production. CARA adds the missing *admission* step that measures the downstream cascade directly before an update is deployed, and its low-cost drift screen (Section III-B) can operate alongside the same production monitors.

VII. RELATED WORK

A. End-to-End Driving and Its Maintenance Limits

UniAD put detection, tracking, mapping, motion forecasting, occupancy prediction, and planning into a single planning-oriented network [2]. VAD replaced dense rasterized scene representations with vectorized ones for efficiency [12]. SparseDrive introduced fully sparse scene representations and reported the resource profile of the flagship systems: $7.2\times$ faster training and $5\times$ faster inference than UniAD, whose training takes about 144 GPU-hours [14]. The main survey of the field lists interpretability, causal confusion, and robustness under distribution shift as open challenges, and it does not treat component-level maintenance or retraining policy at all [1].

A recent end-to-end repair literature addresses maintenance directly. CorrectAD builds a self-correction loop that diagnoses failures, generates targeted training video with a diffusion model, and fine-tunes the entire end-to-end planner on old plus new data, correcting 62.5% of failure cases on nuScenes [20]. R2SE avoids full-model retraining by freezing

the generalist model and training reinforcement-learned low-rank adapter specialists on failure regions, motivated by the catastrophic forgetting that naive fine-tuning causes [21]. Both reintroduce pipeline structure to make maintenance tractable, which motivates the pipeline architecture adopted in this work.

B. Evaluation of Driving Systems

Our planning metric is open-loop displacement error on nuScenes, and the limits of this protocol are well documented. Codevilla et al. showed as early as 2018 that offline prediction error correlates only weakly with actual closed-loop driving quality [15]. Zhai et al. found that a trivial MLP using only ego state, with no perception at all, matches perception-based end-to-end methods on nuScenes open-loop L2 [13]. Dauner et al. showed that open-loop ego-forecasting and closed-loop driving are misaligned tasks, and that a largely rule-based planner, PDM-Closed, outperformed learned planners [10]. Their later NAVSIM benchmark provides short-horizon non-reactive metrics that align far better with closed-loop performance [16]. These critiques target open-loop metrics used to *rank* architectures. Our use is differential (the same unchanged planner, same scenes, one upstream change), and much of that weakness therefore cancels, and we report collision rate alongside L2 and revisit the caveat in Section VI.

C. Model Updates, Regression, and Interface Compatibility

What we measure at system scale has per-sample analogues in the model-update literature. Yan et al. showed that model updates cause negative flips, samples the old model handled correctly that the new model gets wrong, at rates of 4–5% even between equal-accuracy retrains of identical architectures, and they proposed positive-congruent training with focal distillation to suppress them [17]. The backward-compatible training of Shen et al. constrains a new upstream representation model so that unchanged downstream consumers keep working [18], which is the same situation as updating C_1 beneath an unchanged C_2 and C_3 . On the monitoring side, the data validation system of Breck et al. runs two-sample distribution tests between pipeline stages in production [11]. On a nuScenes detector–tracker–predictor stack nearly the same shape as ours, Ivanovic et al. showed that propagating state uncertainty across the perception–prediction interface, instead of passing point estimates, improves forecasting by a wide margin [19]. These works provide mitigation techniques. This paper contributes the complementary controlled measurement of the cascade that such techniques are intended to mitigate.

D. Reliability Modeling of Machine Learning Systems

The technical-debt analysis of Sculley et al. introduced CACE, unstable data dependencies, and correction cascades as qualitative failure modes of ML systems [6]. The base paper for this work, Wang and Machida [4], formalized imperfect retraining of a two-model machine learning system as a continuous-time Markov chain, named entangled enhancement, and compared progressive and conservative

maintenance policies. Their follow-on work casts the same problem as an availability–continuity trade-off [5]. Both are analytical models whose state space grows as 2^N for N components, and both are calibrated on synthetic transition rates. This paper provides an empirical counterpart. The isolation property we verify empirically is the property their state factorization assumes, and the coupling factor we propose (Section III) is a measured statistic that can replace those synthetic rates.

VIII. CONCLUSION

We presented a controlled study of single-model retraining in a perception–prediction–planning pipeline under a geographic domain shift. Using the two-mode measurement with a verified isolation control on the full nuScenes trainval split (2,020 evaluation frames), and running a campaign of 15 retraining updates rather than a single event, we found that retraining C_1 disturbs the downstream plan in every case. The retrained detector improves on its own metric in all 15 updates, and yet in 6 of them the planner’s trajectory-agreement metric worsens at the same time. Entangled enhancement is therefore not only an analytically modeled hazard but an observable one in a real AV pipeline, reached without any data-balancing intervention. The two planning metrics moreover *diverge*. On the reference update the collision rate improved to zero while L2 agreement worsened. Together these results show that model-level validation, and any single-metric check on the system output, is insufficient for safe maintenance of a machine learning system, since each of those six updates would have passed a review of C_1 alone and a review of the collision rate alone. We proposed the CARA admission method for maintaining such pipelines and evaluated its drift screen on a held-out split. The drift screen tracks the magnitude of downstream disturbance (Spearman ≈ 0.56) but does not classify harm on its own (AUROC ≈ 0.48), and it is therefore a prioritization filter for the full admission procedure rather than a standalone decision rule. The broader contribution is methodological. Cascade effects can be measured under exact controls, but only in an architecture that exposes the connections between its models.

REFERENCES

- [1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, “End-to-end autonomous driving: Challenges and frontiers,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [2] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, “Planning-oriented autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [3] R. Wu, C. Guo, A. Hannun, and L. van der Maaten, “Fixes that fail: Self-defeating improvements in machine-learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 11 745–11 756.
- [4] Z. Wang and F. Machida, “Maintaining performance of a machine learning system against imperfect retraining,” in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.
- [5] Z. Wang and F. Machida, “Exploiting the availability–continuity trade-off in imperfect retraining of machine learning systems,” in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.
- [6] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.
- [7] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nusenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.
- [8] G. Jocher and J. Qiu, “Ultralytics yolo11,” <https://github.com/ultralytics/ultralytics>, 2024.
- [9] M. Treiber, A. Hennecke, and D. Helbing, “Congested traffic states in empirical observations and microscopic simulations,” *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.
- [10] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, “Parting with misconceptions about learning-based vehicle motion planning,” in *Conference on Robot Learning (CoRL)*, 2023.
- [11] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data validation for machine learning,” in *Proceedings of Machine Learning and Systems (SysML/MLSys)*, vol. 1, 2019.
- [12] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, “Vad: Vectorized scene representation for efficient autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [13] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nusenes,” *arXiv preprint arXiv:2305.10430*, 2023.
- [14] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, “Sparsedrive: End-to-end autonomous driving via sparse scene representation,” *arXiv preprint arXiv:2405.19620*, 2024.
- [15] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, “On offline evaluation of vision-based driving models,” in *European Conference on Computer Vision (ECCV)*, 2018.
- [16] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, “Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [17] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, “Positive-congruent training: Towards regression-free model updates,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308.
- [18] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, “Towards backward-compatible representation learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377.
- [19] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, “Propagating state uncertainty through trajectory forecasting,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [20] E. Ma, L. Zhou, T. Tang, J. Zhang, J. Jiang, Z. Zhang, D. Han, K. Zhan, X. Zhang, X. Lang, H. Sun, X. Zhou, D. Lin, and K. Yu, “Correctad: A self-correcting agentic system to improve end-to-end planning in autonomous driving,” *arXiv preprint arXiv:2511.13297*, 2025.
- [21] H. Liu, T. Li, H. Yang, L. Chen, C. Wang, K. Guo, H. Tian, H. Li, H. Li, and C. Lv, “Reinforced refinement with self-aware expansion for end-to-end autonomous driving,” *arXiv preprint arXiv:2506.09800*, 2025.